

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process

concepts to model decision-making problems.(BL2,BL3)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

1. Aim

To design a Markov Decision Process (MDP) for an RL-based warehouse robot that learns to pick, pack, and transport items efficiently, while avoiding obstacles, minimizing delays, improving accuracy, and maintaining safety. The experiment also analyzes how the reward function affects the robot's learning and behavior.

2. Objective

The main objectives are:

To model a warehouse robot problem using a **Markov Decision Process**.

To define the **state space, action space, reward function, policy, and environment**.

To design a reward function that balances:

- **Speed**

Accuracy

Safety

To understand how rewards influence the robot's learning behavior.

To encourage the robot to complete warehouse tasks efficiently without collisions or incorrect deliveries.

3. Introduction

Reinforcement Learning (RL) is a machine learning technique in which an **agent learns by interacting with an environment**. The agent observes the current state, takes an action, receives a reward, and moves to a new state.

In this problem, the **warehouse robot is the RL agent**. It must perform tasks such as picking an item from a storage location, packing it correctly, and transporting it to the required destination. The robot must also avoid obstacles, reduce unnecessary movement, and complete tasks quickly.

This problem can be represented using a **Markov Decision Process (MDP)**, which is defined as:

where:

= Set of states

= Set of actions

= State transition probability

= Reward function

= Discount factor

4. Action Space

The **action space** defines the possible actions the robot can perform.

Navigation Actions

Move Up

Move Down

Move Left

Move Right

Stay/Wait

Warehouse Task Actions

Pick Item

Pack Item

Drop/Deliver Item

Example

The robot selects one action according to its current policy.

For example:

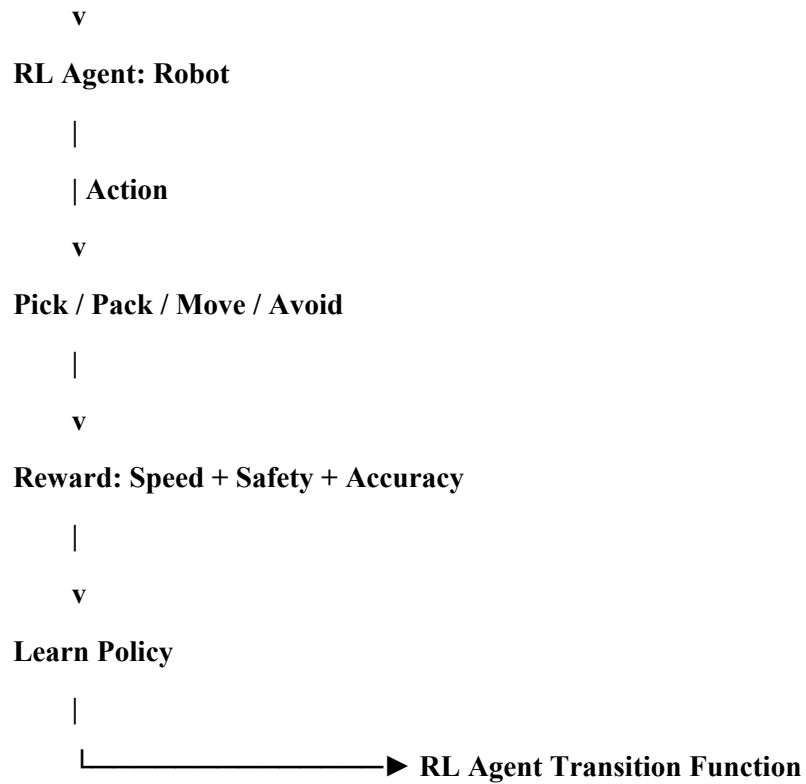
means the robot moves one step to the right.

DIAGRAM:

Environment: Warehouse

|

| **State**



The transition function determines how the warehouse changes after an action.

It represents the probability of reaching the next state after taking action in state .

Example

If the robot is at position (2,3) and chooses Right:

5. Code:-

```
casestudy_1.py > ...
9  class WarehouseEnvironment:
103 def step(self, action):
239
240     if (
241         self.item_picked == 1
242         and self.item_packed == 1
243         and self.item_delivered == 0
244     ):
245
246         self.item_delivered = 1
247
248         # Successful delivery
249         reward = 50
250
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** ... Python + v ... | [] X

```
C:\Users\Madamanchigunashekar\OneDrive\Desktop\case study>C:\Users\Madamanc
higunashekar\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/M
adamanchigunashekar/OneDrive/Desktop/case study/casestudy_1.py"
Training started...
Episode: 500 | Reward: 80.5 | Epsilon: 0.082
Episode: 1000 | Reward: 80.5 | Epsilon: 0.01
Episode: 1500 | Reward: 80.5 | Epsilon: 0.01
Episode: 2000 | Reward: 82.5 | Epsilon: 0.01
Episode: 2500 | Reward: 80.5 | Epsilon: 0.01
Episode: 3000 | Reward: 75.0 | Epsilon: 0.01
Episode: 3500 | Reward: 80.5 | Epsilon: 0.01
Episode: 4000 | Reward: 80.5 | Epsilon: 0.01
Episode: 4500 | Reward: 80.5 | Epsilon: 0.01
Episode: 5000 | Reward: 81.5 | Epsilon: 0.01
```

6. Conclusion

The warehouse robot problem can be effectively modeled as a **Markov Decision Process**, where the robot acts as the agent and the warehouse acts as the environment. The state represents the robot's location, battery, item and task status, while the actions include movement, picking, packing, and delivery. A carefully designed reward function encourages the robot to complete tasks quickly and accurately while avoiding collisions and unsafe actions. The reward structure has a major influence.

2. 1.Aim

To design a Reinforcement Learning (RL)-based movie recommendation system that dynamically recommends movies based on user preferences and viewing history, while analyzing reward signals and the trade-off between exploration and exploitation.

2. Objective

- To define the **state, action, reward, policy, and environment** for a movie recommendation system.
- To design a suitable reward function based on user engagement.
- To analyze how different rewards influence recommendations.
- To understand the balance between **exploration and exploitation**.

3. Introduction

In a movie streaming platform, Reinforcement Learning can be used to recommend movies according to a user's changing interests. The **user is the environment**, while the **recommendation agent** selects movies and learns from user feedback such as watching, skipping, liking, or rating a movie.

The system can be modeled as an MDP:

4.Algorithm: Q-Learning

The recommendation agent can use Q-Learning:

5.Exploration vs Exploitation:-

Exploitation

The system recommends movies that it already knows the user likes.

Advantages:

- Higher immediate engagement.
- Better personalized recommendations.

Disadvantage:

- May repeatedly recommend similar movies.
- User may experience limited content diversity.

Exploration

The system recommends new or less-known movies to discover user interests.

Advantages:

- Finds new preferences.

```
casestudy_2.py > ...
136 # -----
137
138 Q[state, action] = (
139     Q[state, action]
140     + alpha
141     * (
142         reward
143         + gamma * np.max(Q[next_state])
144         - Q[state, action]
145     )
146 )
147
148 # -----

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL ... Python + v ... | [ ] X

=====
Q-Values:
Action_Lover -> [93.3  13.82 17.7  32.87 15.99]
Comedy_Lover -> [29.18 93.57 22.38 10.71 25.86]
Drama_Lover -> [29.95 35.22 94.53 13.79 22.89]

=====
FINAL MOVIE RECOMMENDATIONS
=====
User Preference: Action_Lover
Recommended Movie: Action Movie
Reward Score: 93.3
-----
User Preference: Comedy Lover
```

6. Expected Result

The RL-based recommendation system learns to provide personalized movie recommendations by using user viewing history and feedback. A suitable reward function improves engagement by encouraging movies that users watch, like, and rate positively. A proper balance between exploration and exploitation allows the system to recommend familiar movies while also discovering new user interests.

7. Conclusion

Reinforcement Learning can make movie recommendations more dynamic and personalized by continuously learning from user feedback. A balanced reward function based on watch time, likes, ratings, and skips can improve user engagement. The exploration-exploitation trade-off is important because excessive exploitation limits content discovery, while excessive exploration may reduce immediate engagement. Therefore, an ϵ -greedy strategy provides a practical balance for an effective movie recommendation system.

3. Project: RL-Based Smart Agricultural Irrigation and Fertilizer Optimization

1. Aim

To design an autonomous agricultural system using Reinforcement Learning (RL) that optimizes irrigation and fertilizer usage based on soil conditions and weather forecasts, while maximizing crop yield and minimizing resource wastage.

2. Objectives

- To monitor soil moisture, soil nutrients, temperature, and weather conditions.
- To define suitable states and actions for an agricultural RL environment.
- To design a reward function that balances crop yield and resource consumption.
- To study the effect of delayed rewards on crop yield optimization.

DIAGRAM:-

Soil Sensors

|

Weather Data

|

RL Agent

|

Irrigation + Fertilizer

|

Crop Growth

|

Reward

|

RL Agent Learns

4 .Algorithm

PPO-Based Agricultural Optimization

1. Initialize the RL agent.
2. Initialize the agricultural environment.
3. Read soil and weather conditions.
4. Construct the current state.
5. Select irrigation and fertilizer actions using the policy.
6. Apply the selected actions to the field.
7. Observe the new soil and crop conditions.

5. CODE:-

```
casestudy_3.py > ...
35
36     # Reward
37     if state == 0 and action == 1:
38         reward = 10      # Water dry soil
39         next_state = 1
40
41     elif state == 1 and action == 2:
42         reward = 10      # Fertilize normal soil
43         next_state = 1
44
45     elif state == 2 and action == 1:
46         reward = -10     # Too much water
47         next_state = 2
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** ... Python + v ... | []

```
C:\Users\Madamanchigunashekar\OneDrive\Desktop\case study>C:\Users\Madamanc
higunashekar\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/M
adamanchigunashekar/OneDrive/Desktop/case study/casestudy_3.py"
Learned Q-Table:
[[ 85.31505878 100.          82.47192863]
 [ 89.          89.          100.         ]
 [-9.99999716 -18.99999114 -9.99999716]]

Recommended Actions:
Dry Soil -> Add Water
Normal Soil -> Add Fertilizer
Wet Soil -> No Water
```

6. Conclusion

The proposed RL framework enables an autonomous agricultural system to make intelligent irrigation and fertilizer decisions using real-time soil conditions and weather forecasts. The state representation includes soil moisture, nutrient levels, temperature, humidity, rainfall predictions, and crop growth stage, while the action space controls irrigation and fertilizer levels. Since crop yield is observed only after a long growing period, delayed rewards create a credit-assignment challenge. Combining intermediate rewards for soil and crop health with a final harvest reward helps the agent learn more effectively. PPO is recommended for a realistic system because it can handle complex, uncertain, and continuous agricultural environments. Overall, the framework can improve crop yield while reducing water and fertilizer consumption.

4.Project: RL-Based Real-Time Network Intrusion Detection and Response System

1. Aim

To design a **Reinforcement Learning (RL)-based cybersecurity system** that detects network attacks in real time and selects suitable responses to protect the network from threats.

2. Objectives

- To model network traffic and security conditions as RL states.
- To define the cybersecurity environment for intrusion detection.
- To design suitable actions for responding to network attacks.
- To create a reward function that encourages accurate detection and fast response.
- To analyze the problems of **sparse rewards**.

3. Introduction

Traditional cybersecurity systems often use predefined rules and signatures to detect attacks. However, modern cyberattacks can change their patterns and may not always match known signatures. Reinforcement Learning can help a cybersecurity system learn from network conditions and feedback. The RL agent observes network traffic, identifies possible threats, and chooses an appropriate response. The agent receives rewards for correctly detecting attacks and penalties for incorrect decisions.

DIAGRAM:

Network Traffic

|

Network Monitoring

|

State Representation

|

RL Agent

|

Attack Detection

|

Select Response

|

Network Environment

|

Reward

|

RL Agent Learns

4. Reinforcement Learning Framework

The cybersecurity problem can be modeled as a Markov Decision Process:

5. DQN Advantages

- Handles large state spaces.
- Uses neural networks to approximate Q-values.
- Can learn complex attack patterns.
- Suitable for dynamic network environments.

The DQN agent selects the action with the highest predicted Q-value:

The Q-value is updated using:

6. Algorithm

RL-Based Intrusion Detection and Response

1. Initialize the RL agent.
2. Initialize the network environment.
3. Collect network traffic data.
4. Extract network features.
5. Construct the current state.
6. Select an action using an ϵ -greedy policy.
7. Allow, monitor, block, or isolate traffic.
8. Observe the result.
9. Calculate the reward.
10. Update the RL model.

7. Expected Result

The RL-based cybersecurity system should learn to:

- Identify normal and suspicious network traffic.
- Detect network attacks.
- Block malicious traffic.
- Isolate compromised devices.
- Reduce false alarms.
- Respond quickly to threats

- Code:-

```
5
6 states = [
7     "Normal Traffic",
8     "Suspicious Traffic",
9     "Attack Traffic"
10 ]
11
12 actions = [
13     "Allow Traffic",
14     "Monitor Traffic",
15     "Block Traffic",
16     "Isolate Device"
17 ]
18
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** ... Python + v ... | [] >

```
Network State: Normal Traffic
RL Action: Allow Traffic
Reward: 10
-----
Network State: Suspicious Traffic
RL Action: Monitor Traffic
Reward: 10
-----
Network State: Attack Traffic
RL Action: Block Traffic
Reward: 20
-----
```

8. Conclusion

Reinforcement Learning provides an effective approach for building an adaptive cybersecurity system that can detect and respond to network attacks in real time. The network environment can be represented using traffic features such as packet rates, bandwidth, failed login attempts, active connections, and attack probability. The RL agent can perform actions such as allowing, monitoring, blocking, or isolating network traffic. However, sparse rewards and real-time response requirements are major challenges. Reward shaping can provide more frequent feedback, while DQN can handle complex network states. With an appropriate reward function and exploration-exploitation strategy, the RL agent can learn to improve intrusion detection and respond quickly to cyber threats.

5. Project: RL-Based Driver Allocation and Dynamic Pricing in Ride-Sharing

1. Aim

To design a Reinforcement Learning (RL)-based ride-sharing system that optimizes driver allocation and dynamic pricing according to real-time demand, driver availability, traffic, and environmental conditions.

2. Objectives

- To define suitable states for the ride-sharing environment.
- To define actions for driver allocation and dynamic pricing.
- To design a reward function that balances company profit, passenger waiting time, and driver utilization.
- To analyze how environmental uncertainty affects RL learning.
- To evaluate ethical concerns related to dynamic pricing.

3. Introduction

- A ride-sharing platform needs to efficiently match available drivers with passengers. Demand can change rapidly depending on location, time, traffic, weather, and special events.
- RL can help the platform learn the best decisions by observing the current environment, assigning drivers, setting prices, and receiving rewards based on the results.

4. RL Framework

The ride-sharing problem can be represented as an MDP:

where:

- S = State space
- A = Action space
- P = Transition probability
- R = Reward function
- γ = Discount factor.

DIAGRAM:-

Passenger Demand

|

v

Driver Availability

|

Traffic + Weather

|
RL Agent

|
Driver Allocation + Dynamic Pricing

|
Ride-Sharing Environment

|
Reward

|
RL Agent Learns

5.Algorithm

RL-Based Ride-Sharing Optimization:-

1. Initialize the RL agent.
2. Initialize the ride-sharing environment.
3. Observe passenger demand.
4. Observe available drivers.
5. Collect traffic and weather information.
6. Construct the current state.
7. Select an action using an ϵ -greedy strategy.
8. Allocate drivers to suitable locations.
9. Set an appropriate dynamic price.

6.code

```
60 def get_reward(state, action):
84
85     elif state == 1:
86
87         if action == 0:
88             return 5
89
90         elif action == 1:
91             return 10
92
93         elif action == 2:
94             return 3
95
96         elif action == 3:
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** ... Python + v ... | []

```
Demand: Low Demand
Decision: Normal Price + Keep Drivers
Reward: 10
-----
Demand: Medium Demand
Decision: Normal Price + Keep Drivers
Reward: 5
-----
Demand: High Demand
Decision: Normal Price + Move Drivers
Reward: 15
```

7. Conclusion:-

Reinforcement Learning can be used to optimize driver allocation and dynamic pricing in ride-sharing platforms. The RL agent observes passenger demand, driver availability, traffic, weather, location, and current prices to make decisions. Environmental uncertainty makes learning challenging because demand and traffic can change unexpectedly. The reward function should balance company revenue with passenger waiting time, driver utilization, customer satisfaction, and pricing fairness. Although dynamic pricing can improve supply-demand balance, ethical concerns such as unfair price increases, discrimination, lack of transparency, and emergency pricing must be addressed. Therefore, a well-designed RL system should optimize both operational efficiency and fairness, rather than focusing only on profit.

